

Aula 11 — Revisão da Unidade 01

Um problema novo, as mesmas ideias

DESAFIO

A pergunta da revisão

Se eu entregar um sistema que você nunca viu, você consegue descobrir quem sabe o quê, quem faz o quê, quem conhece quem e o que muda depois de cada operação?

Missão de exploração

- robôs investigam uma região;
- cada robô usa um sensor;
- uma missão reúne participantes e registra observações;
- depois de encerrada, sua estrutura e seus registros são preservados.

Ainda não existem classes.

ATIVIDADE

ATO 1 — O PROBLEMA

Antes do código

1. Que coisas precisam manter estado próprio — e que substantivo não precisa virar classe?
2. Que responsabilidades já aparecem antes de escolher campos e métodos?

Primeiras hipóteses

Sensor , Robot , Leitura e Missao podem ganhar papéis no modelo.

Equipe e região não precisam virar classes neste recorte.

Uma classe se justifica pelo papel de seus objetos, não apenas por um substantivo no enunciado.

PRIMEIRO OBJETO

Um sensor precisa nascer

```
Sensor temperatura =  
    new Sensor("Temperatura", "°C");
```

`new` cria uma identidade; os argumentos fornecem o estado inicial.

ATIVIDADE

PRIMEIRO OBJETO

E se a criação fosse vazia

```
Sensor sensor = new Sensor();  
sensor.tipo = "Temperatura";  
sensor.unidade = "°C";
```

Em que momento o objeto faz sentido, e o que outro objeto observaria se recebesse `sensor` logo após o `new` ?

Existir não significa estar completo

Logo após `new Sensor()` :

```
tipo = null  
unidade = null
```

Os campos de referência possuem valor padrão `null` .

O construtor faz a criação terminar com os dados necessários.

Argumento, parâmetro e campo

```
public Sensor(String tipo, String unidade) {  
    this.tipo = tipo;  
    this.unidade = unidade;  
}
```

"Temperatura" → tipo → this.tipo

`this.tipo` é o campo do objeto atual; `tipo` é o parâmetro recebido.

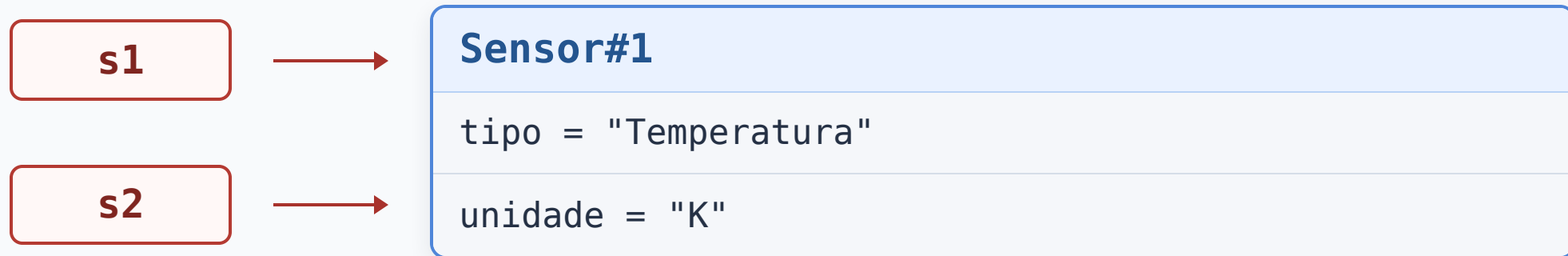
Uma atribuição, nenhuma criação nova

```
Sensor s1 = new Sensor("Temperatura", "°C");  
Sensor s2 = s1;
```

```
// versão exploratória: campo ainda exposto  
s2.unidade = "K";
```

1. Quantas variáveis e quantos objetos existem?
2. O que `s1` observa, o que foi copiado para `s2` e o que `s1 == s2` responderia?

Duas variáveis, um sensor



Duas variáveis. Uma identidade. `s1` observa `"K"`; `s1 == s2` produz `true`.

ATIVIDADE

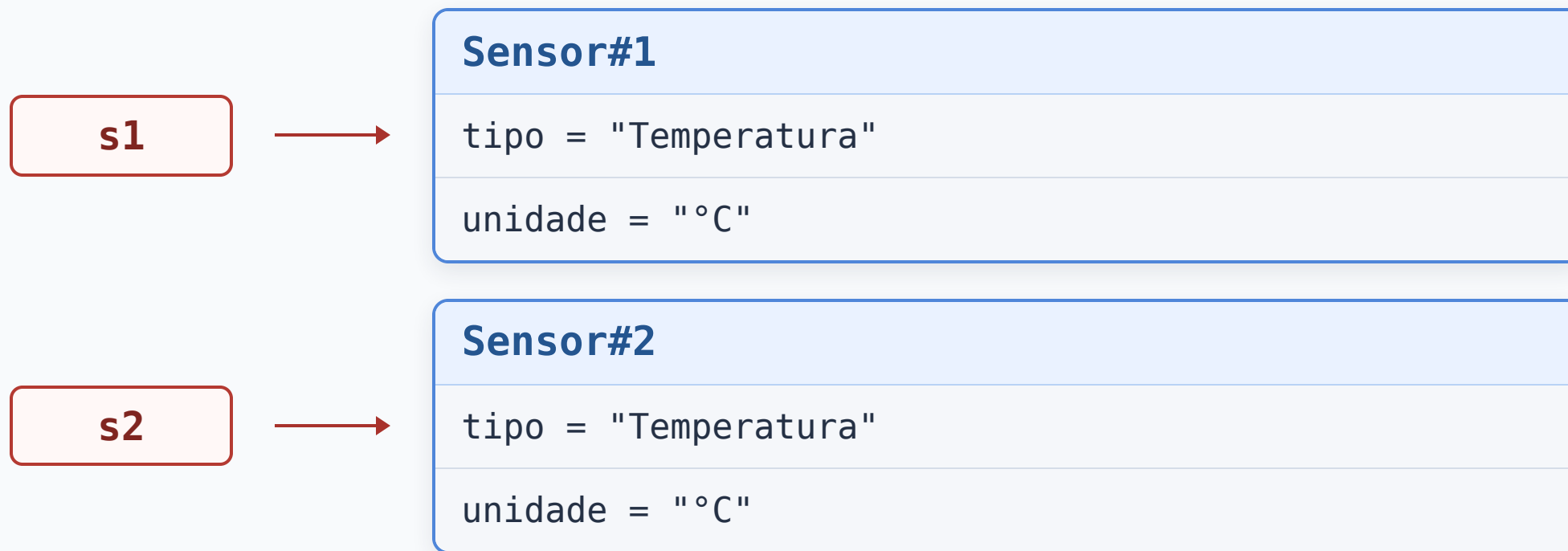
REFERÊNCIAS E IDENTIDADE

Agora há dois new

```
Sensor s1 = new Sensor("Temperatura", "°C");  
Sensor s2 = new Sensor("Temperatura", "°C");
```

Compare o estado observado com a identidade dos dois sensores.

Mesmos valores, objetos distintos



`s1 == s2` produz `false`.

CONCEITO-CHAVE

REFERÊNCIAS E IDENTIDADE

Estado não é identidade

- variável mantém uma referência;
- `new` cria um objeto;
- atribuição entre variáveis copia a referência;
- `==` verifica se duas referências chegam ao mesmo objeto.

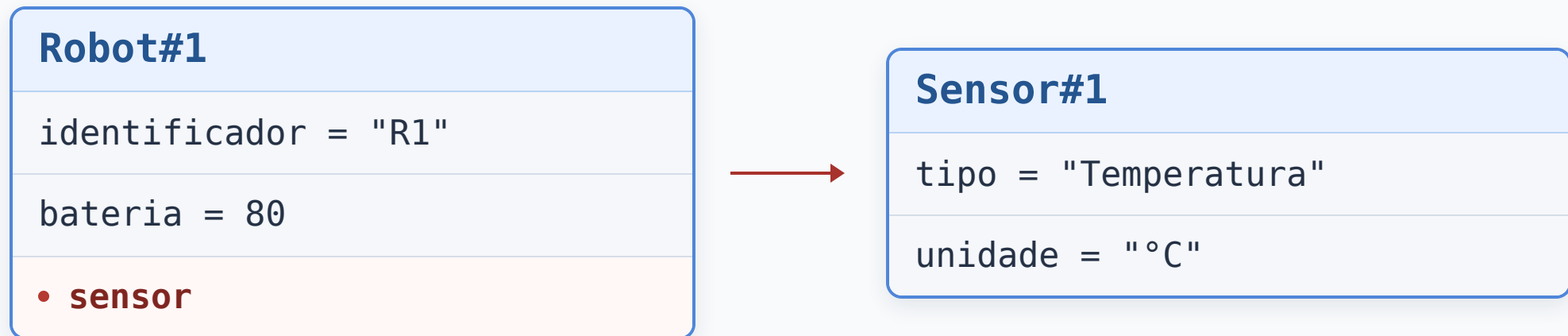
O sensor participa de outro objeto

```
Sensor temperatura =  
    new Sensor("Temperatura", "°C");  
  
Robot r1 =  
    new Robot("R1", 80, temperatura);
```

Acompanhe as referências

1. Quantos objetos foram criados, e o robô contém uma cópia do sensor?
2. Quem conhece quem — e qual direção inversa ainda não é necessária?

Um objeto como argumento



O parâmetro recebeu uma referência. Nenhuma cópia foi criada.

ARMADILHA

ESTADO PROTEGIDO

O Java aceita; o domínio recusa

```
robot.bateria = -20;
```

Um `int` guarda `-20`. Uma bateria do domínio não deveria.

Se qualquer código com uma referência puder escrever o campo, onde está protegida a regra?

ATIVIDADE

ESTADO PROTEGIDO

Duas tentativas

```
robot.deslocar(15);  
robot.deslocar(100);
```

Com bateria inicial 80 , qual operação muda o estado — e por que um `setBateria(...)` irrestrito não resolveria o problema?

Uma operação com intenção

```
public void deslocar(int consumo) {  
    if (consumo > 0 && consumo <= bateria) {  
        bateria -= consumo;  
    }  
}
```

Estado final: `bateria = 65` .

Encapsular é controlar mudanças significativas — não gerar getters e setters automaticamente.

SURGE LEITURA

Uma observação precisa permanecer

```
Leitura leitura = r1.realizarLeitura(23.4);
```

Ainda não decidimos quem cria nem o que esse registro conhece.

ATIVIDADE

SURGE LEITURA

Decida pelas responsabilidades

1. Quem conhece o sensor necessário para criar o registro?
2. O que a leitura precisa preservar, e quem deve manter o histórico?

Uma decisão simples

Leitura preserva:

- referência para o Robot que realizou a ação;
- tipo e unidade observados naquele momento;
- valor registrado.

Robot cria a leitura. Missao manterá o histórico.

O robô colabora com o sensor

```
public Leitura realizarLeitura(double valor) {  
    return new Leitura(  
        this,  
        sensor.getTipo(),  
        sensor.getUnidade(),  
        valor  
    );  
}
```

O objeto solicita ao colaborador apenas as informações necessárias.

Trabalho distribuído

Robot

- conhece o Sensor atual
- solicita tipo e unidade
- cria Leitura

Leitura

- preserva origem + contexto + valor

Nenhum objeto precisa conhecer tudo.

ATIVIDADE

CONSEQUÊNCIAS DAS REFERÊNCIAS

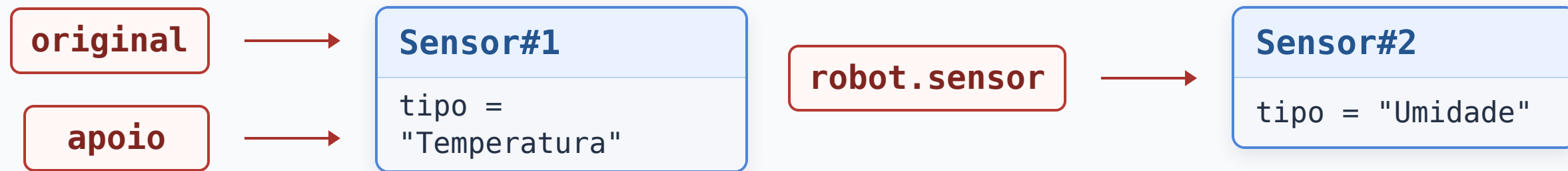
O sensor do robô muda

```
Sensor original = new Sensor("Temperatura", "°C");  
Robot robot = new Robot("R1", 80, original);  
Sensor apoio = original;
```

```
Sensor novoSensor = new Sensor("Umidade", "%");  
robot.trocarSensor(novoSensor);
```

`original == apoio` e `robot.usaSensor(original)` produzem quais resultados?

Trocar um campo não transforma objetos



`true` e `false`.

Uma **Leitura** anterior preserva tipo e unidade; a troca afeta apenas as próximas observações.

Uma decisão sobre referências produz consequências observáveis ao longo do tempo.

A HISTÓRIA CRESCE

Surge Missao

```
Missao missao = new Missao("Vale Norte");  
  
missao.adicionarRobot(r1);  
missao.adicionarRobot(r2);
```

Agora existe um conjunto a coordenar.

ATIVIDADE

SURGE MISSAO

Quem mantém a equipe

Defina quem deve guardar os participantes e justifique por que cada robô não precisa manter uma lista de missões.

Duas coleções, dois papéis

```
private List<Robot> robots;  
private List<Leitura> leituras;  
  
public Missao(String nome) {  
    this.nome = nome;  
    robots = new ArrayList<>();  
    leituras = new ArrayList<>();  
    encerrada = false;  
}
```

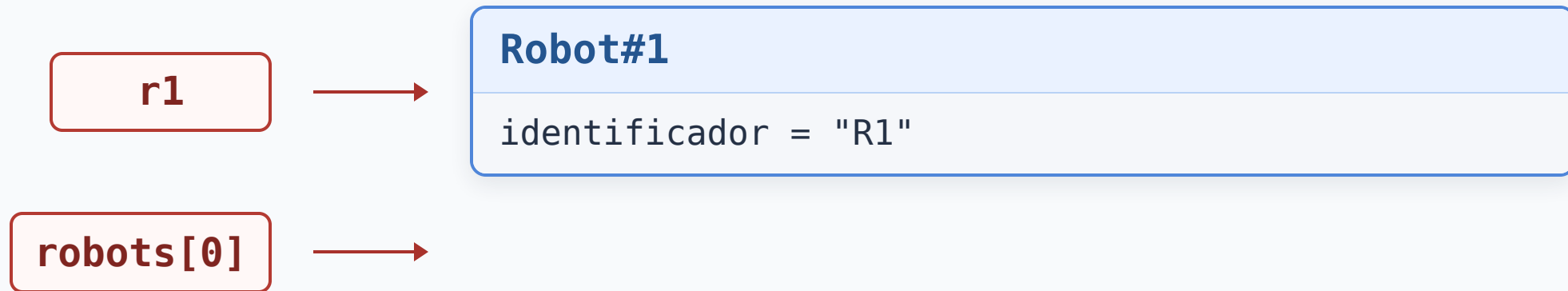
Participantes existem antes da missão; leituras surgem durante ela.

Adicionar mantém uma referência

```
public void adicionarRobot(Robot robot) {  
    if (!encerrada && !participa(robot)) {  
        robots.add(robot);  
    }  
}
```

add não executa new Robot() .

Uma coleção não copia seus elementos



A variável externa e a posição da lista chegam à mesma identidade.

REGISTRAR LEITURAS

Uma nova operação integra o modelo

```
missao.registrarLeitura(r1, 23.4);
```

A chamada envolve participação, criação, colaboração e armazenamento.

ATIVIDADE

REGISTRAR LEITURAS

Antes da implementação

1. Como a missão sabe se o robô participa?
2. Quem cria a leitura — e em que ordem verificamos, criamos e guardamos?

Coordenação sem invadir o robô

```
public void registrarLeitura(Robot robot, double valor) {  
    if (!encerrada && participa(robot)) {  
        Leitura leitura = robot.realizarLeitura(valor);  
        leituras.add(leitura);  
    }  
}
```

Missao protege a regra; Robot produz o registro.

ATIVIDADE

REGISTRAR LEITURAS

A ordem também conta objetos

Decida se uma missão encerrada deve criar uma `Leitura` antes de recusar o registro.

A verificação vem antes da criação

Uma tentativa recusada:

- não cria `Leitura` ;
- não altera a lista;
- não pede trabalho ao robô.

Preservar o estado pode preservar também a quantidade de objetos criados.

Desenhe somente o que os objetos mantêm

1. Quais referências existem depois do primeiro registro?
2. Qual seta seria desnecessária para as responsabilidades atuais?

O grafo necessário

Missao#1

- robots[0]

Robot#1

- sensor

Sensor#1

tipo = "Temperatura"

Missao#1

- leituras[0]

Leitura#1

- robot

Direção exige justificativa

Robot precisa conhecer Missao, e Sensor precisa guardar as leituras que ajudou a produzir?

Conhecimento mínimo

- Robot → Sensor : produzir observações e trocar o componente;
- Leitura → Robot : preservar a origem;
- Missao → Robot : coordenar participantes;
- Missao → Leitura : preservar registros.

A relação inversa só entra quando uma responsabilidade a exige.

ATIVIDADE
COMPOSIÇÃO

Nem toda relação tem o mesmo papel

Qual relação representa participantes independentes, e qual representa partes estruturais preservadas pelo todo?

Composição como responsabilidade estrutural

- Missao → Robot : associação com participantes independentes;
- Robot → Sensor : componente da configuração atual do robô;
- Missao → Leitura : registros que formam a estrutura histórica da missão.

“Possuir uma referência” não basta para chamar toda relação de composição.

NOVA REGRA

Missão aberta e encerrada

Depois de `missao.encerrar()` :

- não entram nem saem participantes;
- não há troca de sensor pela missão;
- não surgem novas leituras;
- os registros existentes permanecem.

ATIVIDADE

NOVA REGRA

Localize a proteção

Quem deve manter o estado encerrado e impedir as mudanças, sem fazer **Robot** conhecer **Missao** ?

Cada missão controla sua transição

```
private boolean encerrada;  
  
public void encerrar() {  
    encerrada = true;  
}
```

As operações de edição consultam esse campo antes de mudar as coleções ou pedir trabalho a um robô.

ESTADO POR INSTÂNCIA

Duas missões, dois estados

```
Missao norte = new Missao("Vale Norte");  
Missao sul = new Missao("Vale Sul");  
  
norte.encerrar();  
sul.adicionarRobot(r1);
```

Fechar `norte` não altera `sul`.

ATIVIDADE

CRIAÇÃO EVITADA

Depois de encerrar

```
missao.registrarLeitura(r1, 25.0);
```

Quantas leituras são efetivamente criadas, e qual estado muda?

CRIAÇÃO EVITADA

Nenhuma mudança

```
novas Leitura = 0  
participantes = iguais  
registros = iguais
```

A guarda executa antes de `robot.realizarLeitura(...)`.

ATIVIDADE

REMOVER PARTICIPANTES

Não entregue a coleção

```
missao.removeRobot(r1);
```

Por que `Missao` deve localizar e remover o participante, em vez de entregar sua coleção ao `Main` ?

O dono da coleção coordena

```
public void removerRobot(Robot procurado) {  
    if (!encerrada) {  
        for (int i = 0; i < robots.size(); i++) {  
            if (robots.get(i) == procurado) {  
                robots.remove(i);  
                return;  
            }  
        }  
    }  
}
```

Leituras antigas não são apagadas.

ATIVIDADE

ALTERAR UMA PARTE

A missão troca o sensor

```
missao.trocarSensor(r1, novoSensor);
```

Quem localiza o robô, e quem realmente altera o campo `sensor` ?

Localizar e delegar

```
public void trocarSensor(Robot procurado,
                        Sensor novoSensor) {
    if (!encerrada) {
        for (Robot robot : robots) {
            if (robot == procurado) {
                robot.trocarSensor(novoSensor);
                return;
            }
        }
    }
}
```

Missao localiza; Robot protege o próprio estado.

Outro robô com o mesmo texto

```
Robot r1 = new Robot("R1", 80, sensor);  
missao.adicionarRobot(r1);
```

```
Robot outroR1 = new Robot("R1", 80, sensor);  
missao.removerRobot(outroR1);
```

A busca por referência encontra o participante?

IDENTIDADE REAPARECE

Mesmo identificador não é a mesma identidade

```
r1 == outroR1 → false
```

A missão continua com `r1`.

Não precisamos introduzir `equals()` nem `hashCode()` para a regra desta versão.

Quatro `main` independentes

Agora cada bloco começa do zero.

Consultas como `quantidadeRobots()`, `quantidadeLeituras()` e `estaEncerrada()` devolvem valores — não as coleções internas.

Preveja a saída, desenhe o estado final e explique pelas responsabilidades — não apenas pela sintaxe.

Main 1

```
Sensor a = new Sensor("Temperatura", "°C");  
Sensor b = a;  
Sensor c = new Sensor("Temperatura", "°C");  
  
System.out.println(a == b);  
System.out.println(a == c);  
System.out.println(b.getUnidade());
```

1. O que será impresso?
2. Quantos objetos existem?
3. Quais variáveis apontam para o mesmo objeto?
4. Desenhe o estado final.
5. Por que o resultado ocorre?

Duas criações, três variáveis

```
true  
false  
°C
```

a e b chegam a Sensor#1 ; c chega a Sensor#2 .

Main 2

```
Sensor sensor = new Sensor("Temperatura", "°C");  
Robot robot = new Robot("R1", 60, sensor);  
Robot apoio = robot;  
robot.deslocar(15);  
apoio.deslocar(80);  
apoio.deslocar(-5);  
System.out.println(robot.getBateria());  
System.out.println(robot == apoio);
```

1. Qual é o estado inicial?
2. Quais operações produzem mudança?
3. Qual operação preserva o estado?
4. Por quê?
5. Qual regra está sendo protegida?

Apenas um deslocamento é aceito

```
45  
true
```

O consumo `80` não cabe na carga atual; `-5` não é positivo.

As duas variáveis chegam ao mesmo robô.

Main 3

```
Sensor t = new Sensor("Temperatura", "°C");
Sensor u = new Sensor("Umidade", "%");
Robot r1 = new Robot("R1", 80, t);
Robot r2 = new Robot("R2", 70, u);

Missao m = new Missao("Vale Norte");
m.adicionarRobot(r1);
m.adicionarRobot(r2);
m.registrarLeitura(r1, 23.4);
m.registrarLeitura(r2, 61.0);

System.out.println(m.quantidadeRobots());
System.out.println(m.quantidadeLeituras());
System.out.println(m.quantidadeLeiturasDe(r1));
```

ATIVIDADE

MAIN 3 — COLABORAÇÃO E COLEÇÕES

Explique a execução

1. Quais linhas serão impressas?
2. Quantos objetos existem?
3. Quem colaborou para produzir cada resultado?
4. Quem não precisou conhecer a missão?
5. Quais relações existem ao final?

Duas equipes na memória, um fluxo de colaboração

```
2  
2  
1
```

Nove objetos relevantes: 7 do domínio + 2 listas internas. Objetos `String` não entram nesta contagem.

`Missao → Robot → Sensor` participa da criação; `Missao → Leitura → Robot` preserva o registro.

Main 4 — preparação

```
Sensor t = new Sensor("Temperatura", "°C");
Sensor u = new Sensor("Umidade", "%");
Robot r1 = new Robot("R1", 80, t);
Robot r2 = new Robot("R2", 70, t);

Missao m = new Missao("Vale Norte");
m.adicionarRobot(r1);
m.adicionarRobot(r2);
m.registrarLeitura(r1, 22.0);
m.trocarSensor(r2, u);
m.registrarLeitura(r2, 58.0);
m.removerRobot(r1);
m.encerrar();
```


Main 4 — depois do encerramento

```
m.registrarLeitura(r2, 60.0);  
m.adicionarRobot(r1);  
m.trocarSensor(r2, t);  
  
System.out.println(m.quantidadeRobots());  
System.out.println(m.quantidadeLeituras());  
System.out.println(m.quantidadeLeiturasDe(r1));  
System.out.println(r2.usaSensor(u));  
System.out.println(m.estaEncerrada());
```

Explique o sistema inteiro

1. Quais linhas serão impressas?
2. Qual é o estado final da missão?
3. Quais operações realmente produziram mudanças?
4. Quais foram recusadas?
5. Quantos objetos foram efetivamente criados?
6. Que objetos continuam compartilhados por referência?
7. Quem preservou cada regra?
8. Que colaboração produziu o resultado final?

Estado final preservado

```
1  
2  
1  
true  
true
```

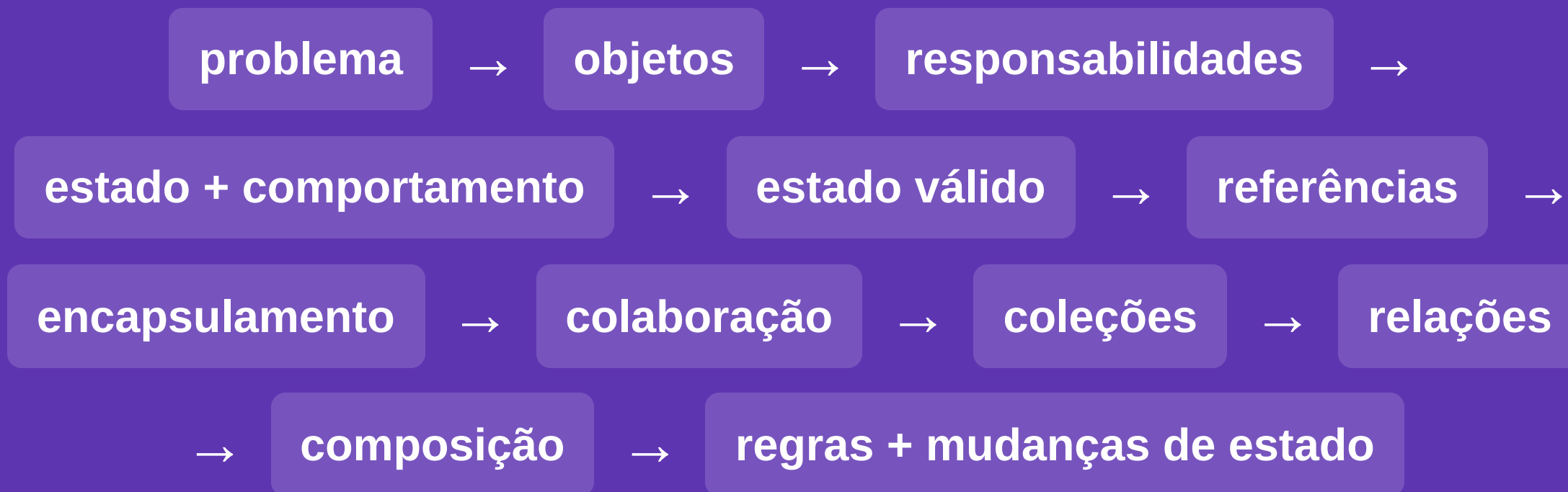
- participante: somente `r2` ;
- registros: duas leituras, uma de cada robô;
- objetos: 2 sensores + 2 robôs + 1 missão + 2 listas + 2 leituras = **9**;
- as três operações posteriores ao encerramento não mudam nem criam objetos.

ATIVIDADE
TRANSFERÊNCIA

Sem robôs agora

1. Um campeonato encerrado recebe novas partidas: quem protege a regra?
2. A verificação ocorre antes ou depois de criar o registro da partida?
3. Um livro precisa guardar todos os empréstimos apenas para fornecer seu título?

A trajetória que reconstruímos



Objetos mantêm seu estado, colaboram por referências e protegem as regras das estruturas que controlam.

Amanhã, outro sistema

Você consegue descobrir quem sabe o quê, quem faz o quê, quem conhece quem e o que acontece depois de cada operação?